

SQL Fundamentals: Introduction, Data Types, Constraints, and Commands

Introduction to SQL

When we mention **SQL**, it's impossible to avoid discussing **data**, as SQL is used to work with and manipulate data in databases.

What is **Data** in SQL?

In SQL, **data** refers to the information stored within a **database**. This data can consist of various types of values, such as: **Numbers, Text, Dates, Binary data**

SQL is a **structured query language** used to manage and manipulate this data, allowing us to **add, update, retrieve, and delete** it.

The term "structured" in SQL refers to how it is specifically designed to handle data in a **table format**. This structured format makes SQL well-suited for working with data stored in tables.

What is a **Database**?

Now that we know SQL handles data then what is database? A **database** is a medium or a place where we store data in a **systematic** and **organized** manner, ensuring that it is easily retrievable and manageable.

Basic Operations in a Database (**CRUD**)

SQL allows us to perform the following operations on data, often referred to as **CRUD** operations:

1. **Create** (Insert) – Add new data into the database.
2. **Read** (Retrieve) – Query and retrieve data from database.
3. **Update** (Modify) – Change existing data in the database.
4. **Delete** (Drop) – Remove data from the database.

These operations ensure that we can **maintain** and **manage** the data in the database effectively.

What is a **Database Management System (DBMS)**?

A **Database Management System (DBMS)** is a **software** or set of programs designed to help maintain and manage databases. It provides a way to store, retrieve, update, and manage the data efficiently.

Key features of a DBMS include:

- **Security** – Protects data from unauthorized access.
- **Authorization** – Grants permissions to authorized users based on roles.

SQL acts as the **query language** that communicates with the DBMS to perform database operations.

A DBMS that follows the relational model (with data organized in tables) is specifically called a **Relational Database Management System (RDBMS)**.

What is a **Table**?

A **table** is a logical organization of data in a relational database. It consists of **rows** and **columns**.

- **Rows**: Horizontal structure used to represent all the properties of a single record or entity.
- **Columns**: Vertical structure used to represent the properties of all the records or entities (e.g., names, ages, addresses).

Let's take a simple example to visualize:

Name	Age
Yogesh	28
Siva	24
Prakash	26

Here, "Yogesh", "Siva", and "Prakash" are records (rows), and **Name** and **Age** are attributes (columns).

The intersection of a row and a column creates **cells**, which hold the actual data.

What is **NoSQL**?

In contrast to SQL, we also hear the term **NoSQL**.

- **NoSQL** is an unstructured database system that stores data in formats like **Graph**, **Key-Value**, **Document**, etc.

Key Differences between **SQL** and **NoSQL**:

- **SQL** is used for **relational databases**.
- **NoSQL** is used for **non-relational databases**.

SQL databases are **structured**, while NoSQL databases are more **flexible** and better suited for unstructured data.

Datatypes in SQL

In SQL, the **datatype** determines the type of data a column can hold. Some common SQL datatypes are:

1. **CHAR**
 2. **VARCHAR**
 3. **DATE**
 4. **NUMBER**
 5. **CLOB**
 6. **BLOB**
-

1. **CHAR** Data Type

- **CHAR** is used for storing fixed-length character strings.

Syntax:

```
CHAR(size)
```

Where **size** specifies the length of the string.

Example:

```
CHAR(10)
```

- **fixed-length** means it will always reserve **10 characters** worth of space in memory, even if fewer characters are used. For example, a string of "YOGESH" would take up 10 characters, with the remaining space being unused (leading to memory wastage).

Y	O	G	E	S	H				
---	---	---	---	---	---	--	--	--	--

2. **VARCHAR** Data Type

- **VARCHAR** is similar to **CHAR**, but it is more **memory efficient**. It stores **variable-length** strings, only allocating the amount of space needed for the actual data.

Syntax:

```
VARCHAR(size)
```

Example:

```
VARCHAR(10)
```

- **variable-length** means if the string "Yogesh" is stored in a column of type `VARCHAR(10)`, it will take up only 6 characters worth of space, rather than 10, avoiding memory wastage.

Y	O	G	E	S	H
---	---	---	---	---	---

3. **DATE** Data Type

- **DATE** is used to store date values in a specified format. The standard formats in **Oracle** are:

Format 1: DD-Mon-YY

Example: 08-Jan-97

Format 2: DD-Mon-YYYY

Example: 02-Mar-1992

4. **NUMBER** Data Type

- The **NUMBER** datatype is used for numeric values and accepts two parameters: **precision** and **scale**.
- **Precision:** The total number of digits the number can have.
- **Scale:** The number of digits to the right of the decimal point.

Example:

```
NUMBER(4)    -- Range: -9999 to +9999
NUMBER(5,2)  -- Range: -999.99 to +999.99
```

If the precision is less than or equal to the scale, the range starts from 0.

Example:

```
NUMBER(2,5)  -- Range: 0.000 to 99.999
```

5. **CLOB** Data Type

- **CLOB** stands for **Character Large Object** and is used to store **large amounts of character data** (up to **4GB**).

6. **BLOB** Data Type

- **BLOB** stands for **Binary Large Object** and is used to store **binary data** such as images, audio files, and videos (also up to **4GB**).

Constraints in SQL

Constraints are rules applied to columns or tables to ensure the data is valid and follows specific conditions. Common constraints include:

1. **UNIQUE**
2. **NOT NULL**
3. **CHECK**
4. **PRIMARY KEY**
5. **FOREIGN KEY**

1. **UNIQUE** Constraint

- The **UNIQUE** constraint ensures that no two rows in a column have the same value.

Syntax:

```
UNIQUE(column_name)
```

Example:

```
UNIQUE(Email ID)
```

Name	Email ID
Yogesh	yogesh@gmail.com
Siva	siva@gmail.com
Yogesh	yogi@gmail.com

In this table, the `Email ID` column has a **UNIQUE** constraint. This means that no two rows can have the same email address. If you try to insert another row with an email that's already in the table, it will raise an error.

2. **NOT NULL** Constraint

- The **NOT NULL** constraint ensures that a column must have a value and cannot be left empty (**NULL**).

What is NULL?

NULL represents an empty or undefined value. It is not the same as `0` or an empty string `' '`. **NULL** does not occupy any memory, and any operation performed on **NULL** results in **NULL**.

Syntax:

```
NOT NULL(column_name)
```

Example:

Name Varchar(100) NOT NULL

Name	Email ID
Yogesh	yogesh@gmail.com
Siva	siva@gmail.com
Yogesh	yogi@gmail.com

If you try to insert a row without specifying a value for the Name, it will throw an error.

3. CHECK Constraint

- The **CHECK** constraint allows you to specify a condition for a column. If the condition is not met, the data is not inserted into the table.

Syntax:

CHECK (age > 20)

Example:

Age INT CHECK (age > 20)

Name	Age
Yogesh	28
Siva	24
Prakash	26

If you try to insert a row with an age less than or equal to 20, it will throw an error

4. PRIMARY KEY Constraint

5. FOREIGN KEY Constraint

The concepts of Primary Key and Foreign Key will be covered separately

SQL Commands

SQL is used to interact with data in a database using queries (commands). SQL commands are categorized into five types:

- **Data Definition Language (DDL)**
 - **Data Manipulation Language (DML)**
 - **Transaction Control Language (TCL)**
 - **Data Control Language (DCL)**
 - **Data Query Language (DQL)**
-

Data Definition Language (DDL)

What is **DDL**?

- SQL is structured, meaning we work with relational databases in a tabular format.
 - DDL is used to **define** tables where data will be stored (e.g., Name, Age, etc.).
 - It includes commands to **create**, **alter**, **drop**, and **truncate** tables.
-

1. **CREATE** – Create a New Table

Step 1: Create a Schema (Database)

```
CREATE SCHEMA Banking;
```

Step 2: Select the Schema

```
USE Banking;
```

Step 3: Create a Transactions Table

```
CREATE TABLE Transactions (  
    TransactionID INT PRIMARY KEY,  
    AccountNumber VARCHAR(20),  
    Amount DECIMAL(10,2),  
    TransactionDate DATE  
);
```

Schema Structure After Creating the Table:

- **Banking**
 - Tables
 - Transactions
 - Columns: TransactionID, AccountNumber, Amount, TransactionDate
 - Indexes, Foreign Keys, Triggers
 - Views
 - Stored Procedures
 - Functions
-

2. **ALTER** – Modify an Existing Table

Add a New Column

```
ALTER TABLE Transactions ADD COLUMN TransactionType VARCHAR(50);
```

Rename Column Amount to TransactionAmount

```
ALTER TABLE Transactions CHANGE Amount TransactionAmount DECIMAL(10,2);
```

Change Data Type of TransactionAmount Column

```
ALTER TABLE Transactions MODIFY COLUMN TransactionAmount FLOAT;
```

3. **DROP** – Delete the Table

```
DROP TABLE Transactions;
```

4. **TRUNCATE** – Delete Data Without Dropping Table

```
TRUNCATE TABLE Transactions;
```

Data Manipulation Language (DML)

DML is used to **add**, **update**, and **delete** data in an existing table.

1. **Insert** Data into the Table

```
INSERT INTO Orders VALUES ('Laptop', 50000);  
INSERT INTO Orders VALUES ('Mobile', 15000);  
INSERT INTO Orders VALUES ('Headphones', 2000);
```

Output:

ProductName	Price
Laptop	50000
Mobile	15000
Headphones	2000

2. Update Data in the Table

(i) Update the Price of Laptop to 55000

```
UPDATE Orders SET Price = 55000 WHERE ProductName = 'Laptop';
```

(ii) Change "Headphones" to "Earbuds"

```
UPDATE Orders SET ProductName = 'Earbuds' WHERE ProductName = 'Headphones';
```

Before Update:

ProductName	Price
Laptop	50000
Mobile	15000
Headphones	2000

After Update:

ProductName	Price
Laptop	55000
Mobile	15000
Earbuds	2000

3. Delete Data from the Table

(i) Delete a Specific Product Record (Laptop)

```
DELETE FROM Orders WHERE ProductName = 'Laptop';
```

(ii) Delete All Rows from the Table

```
DELETE FROM Orders;
```

Data Query Language (DQL)

DQL is used to **retrieve** data using the `SELECT` statement.

1. Retrieve All Data from a Table

```
SELECT * FROM Orders;
```

Output:

ProductName	Price
Laptop	50000
Mobile	15000

2. Retrieve Specific Columns

```
SELECT ProductName FROM Orders;
```

Output:

ProductName
Laptop
Mobile

3. Filter Data Based on Conditions

Retrieve Orders where Price is Greater Than 10,000:

```
SELECT * FROM Orders WHERE Price > 10000;
```

Output:

ProductName	Price
Laptop	50000
Mobile	15000

Transaction Control Language (TCL)

TCL is used to manage **transactions** in SQL. A transaction is a sequence of operations performed as a single logical unit of work.

Key TCL Commands:

1. **COMMIT** – Save Changes Permanently

```
INSERT INTO Orders VALUES ('Tablet', 25000);  
COMMIT;
```

This ensures the new record is permanently stored.

2. **ROLLBACK** – Revert Changes if Not Committed

```
DELETE FROM Orders WHERE ProductName = 'Tablet';  
ROLLBACK;
```

If executed before COMMIT, the deleted record is restored.

3. **SAVEPOINT** – Create a Checkpoint

```
SAVEPOINT Save1;  
UPDATE Orders SET Price = 30000 WHERE ProductName = 'Mobile';  
ROLLBACK TO Save1;
```

This reverts changes only up to the savepoint.

Data Control Language (DCL)

DCL is used to control **access** to the database by granting and revoking permissions.

Key DCL Commands:

1. **GRANT** – Give User Privileges

```
GRANT SELECT, INSERT ON Orders TO user1;
```

user1 can now read and insert data into the Orders table.

2. **REVOKE** – Remove Privileges

```
REVOKE INSERT ON Orders FROM user1;
```

user1 can no longer insert data into the Orders table.